1 Lesson 07-10-26

1.1 ARM hardware interface

The ARM hardware interface is split in 2 components:

- The **CMDSDK** (*Cortex-M System Design Kit*): it is an hardware specification of ready-made peripheral designs that chip designers place next to the CPU;
- The **CMSIS** (*Cortex-M Software Interface Standard*): it is a standard for the C headers and functions that describe a chip.

Our board's peripherals are CMSDK blocks: Arm documents their registers once, the same on every chip that uses them. More specifically, it is an *MPS2*: an Arm development board built around an FPGA (a chip whose logic is loaded at power-on). *AN385* (Application Note 385) is one FPGA image for it: a Cortex-M3 plus CMSDK peripherals.

Address	Device
`0x4000_0000 / 0x4000_1000 / 0x4000_1000`	Timer 0, Timer 1, dual timer
`0x4000_4000 / 0x4000_5000 / 0x4000_6000`	UART0, UART1, UART2
`0x4000_8000 / 0x4001_0000`	Watchdog, GPIO pins
`0x4002_8000`	FPGA I/O (MPS2 block)

We use no vendor CMSIS files (no libraries at all): our register header is hand-written in CMSIS style, so its names look like those in vendor code.

Basically, we have a CMDSDK peripheral set, for which we program a CMSIS interface by hand.

1.1.1 Buses

A **bus** is the set of wires (address, data, control) that carries every load and store from the CPU to a memory or a device. Arm defines two kinds:

- **AHB** (*Advanced High-performance Bus*): the main bus; pipelined, one transfer per cycle. Memories and devices that need speed sit here;
- **APB** (*Advanced Peripheral Bus*): simple, slow, low power. For registers that are touched rarely: configure a timer, send one byte

The bridge turns an AHB access into an APB access: a few extra cycles are needed for conversion. For our code, nothing changes: a register is a load or store at an address on either bus; only the timing differs.

1.1.2 External interrupts

A device can raise an interrupt request (**IRQ**) line to tell the CPU "something happened". Device line N is served by vector-table slot $16 + N$, right after the 16 core slots we saw were related to internal interrupts (exceptions, such as the Reset Handler).

IRQ	Interrupt source	IRQ	Interrupt source
0	UART 0 receive	16	GPIO 2 combined
1	UART 0 transmit	17	GPIO 3 combined
2	UART 1 receive	18	UART 3 receive
3	UART 1 transmit	19	UART 3 transmit
4	UART 2 receive	20	UART 4 receive
5	UART 2 transmit	21	UART 4 transmit
6	GPIO 0 combined	22	SPI 2
7	GPIO 1 combined	23	SPI 3, SPI 4
8	Timer 0	24	GPIO 0, pin 0
9	Timer 1	25	GPIO 0, pin 1
10	Dual Timer	26	GPIO 0, pin 2
11	SPI 0, SPI 1	27	GPIO 0, pin 3
12	UART overflow (UART 0, 1, 2)	28	GPIO 0, pin 4
13	Ethernet	29	GPIO 0, pin 5
14	Audio I2S	30	GPIO 0, pin 6
15	Touch Screen Controller	31	GPIO 0, pin 7

1.2 ARM ABI

Let's look at the ARM **ABI** (*Application Binary Interface*), as it will be useful to know how to implement interrupt handlers (we have to know how arguments are passed around).

C and assembly can call each other only if both follow the same rules: the **AAPCS** (*Procedure Call Standard for the Arm Architecture*).

Register	Role in a call	Kept across the call?
r0–r3	The first four arguments; r0 also carries the result	No: **caller-saved** (scratch)
r4–r11	The function's own long-lived values	Yes: **callee-saved**
r12	Scratch	No: **caller-saved**
lr	Return address, written by `bl`	No: a function that calls another saves it first
sp	Must be a multiple of 8 at every call	Yes

- **Caller-saved**: if the caller still needs r0–r3 or r12 after the call, it saves them itself;
- **Callee-saved**: a function that wants to modify r4 must restore it before returning.

1.3 Memory management and protection